

Figure 1: Electron – major characteristics

the source code to get a better understanding of the features of Electron (a screenshot from the official GitHub page of the source is shown in Figure 3).

The application structure

As stated earlier, Electron allows us to build desktop applications with Web technologies. Primarily, Electron facilitates the building of desktop applications with JavaScript as the core language. It provides a runtime with rich native operating system APIs.

The basic structure of the simple Electron app consists of three files inside your application's folder:

- Package.json
- Main.js
- Index.html

The structure of the *package.json* is the same as Node's modules. The script specified in the 'main' field is the application's startup script, which would run the main process. A sample *package.json* file is shown below:

```
{
  "name" : "your-app",
  "version" : "0.1.0",
  "main" : "main.js"
}
```

If the main field is left blank in the *package.json* file, then Electron tries to load a .js file named *index.js*.

There are two major types of processes in an Electron app, as listed below.

- **Main process:** This is responsible for running the *package.json*'s main script. This main process can display a GUI by building Web pages.
- **Renderer process:** Electron uses Chromium for rendering Web pages. The multi-process architecture of Chromium is also used. These individual GUI (Web) pages also run their own process, which is called the renderer process.

There is a major difference between the normal Web

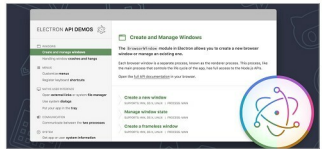


Figure 3: Electron API demos

pages that run inside a browser and the Electron app's Web pages. The normal Web pages are executed in a sand-boxed environment and hence they are denied access to the native system resources. However, Electron Web pages can be used to access the lower level resources of the system with the help of operating system interactions.

The *main.js* file (which is indicated in the *package.json* file) needs to build windows and handle events. A sample *main.js* file is shown below:

```
const {app, BrowserWindow} = require('electron')
const path = require('path')
const url = require('url')

// Keep a global reference of the window object, if you
// don't, the window will be closed automatically when the
// JavaScript object is garbage collected.

let win

function createWindow ()
{
  // Create the browser window.
  win = new BrowserWindow({width: 800, height: 600})

  // and load the index.html of the app.
  win.loadURL(url.format({
    pathname: path.join(__dirname, 'index.html'),
    protocol: 'file:',
    slashes: true
  }))
  // Open the DevTools.
  win.webContents.openDevTools()

  // Emitted when the window is closed.
  win.on('closed', () => {
    win = null
  })

  // This method will be called when Electron has finished
  // initialization and is ready to create browser windows. Some
  // APIs can only be used after this event occurs.
  app.on('ready', createWindow)
```

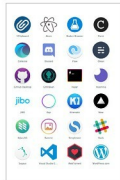


Figure 2: Electron – popular apps from the home page